

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Ingeniería de Software				
TÍTULO DE LA PRÁCTICA:	Título de la Práctica				
NÚMERO DE LA PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	15/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Nombre del Docente					

RESULTADOS Y PRUEBAS
EJERCICIOS/PROBLEMAS PROPUESTOS Ejercicio 1: Agrega una tercera pantalla, HomeScreen(usuario), registrada en el NavHost con la ruta "home", que reciba el nombre del usuario como argumento de navegación (por ejemplo navController.navigate("home/usuario")) y lo muestre en el mensaje "Bienvenido". En este ejercicio se agrega una tercera pantalla a la aplicación de login/registro. La pantalla, llamada HomeScreen(usuario), se registra en el NavHost con la ruta home/{usuario} y recibe el nombre del usuario como argumento de navegación para mostrarlo en un mensaje 1. Definir la lista de cuentas registradas Para validar el ingreso se mantiene una lista global de cuentas. Por ahora contiene el usuario admin con contraseña 1234. Cuando un usuario se registra, la pareja usuario to password se agrega a esta lista: <pre>val cuentasRegistradas = mutableListOf("admin" to "1234")</pre> 2. Registrar la ruta home/{usuario} en el NavHost Dentro del NavHost, cuya ruta inicial es login, se registran tres pantallas: login, registro y home/{usuario}. La ruta de home recibe el argumento usuario de tipo StringType, declarado con navArgument. El valor se obtiene desde backStackEntry.arguments?.getString("usuario") y se pasa al Composable HomeScreen:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
composable( route = "login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            navController.navigate( route = "home/$usuario")
        },
        onIrARegistro = {
            navController.navigate( route = "registro")
        }
    )
}
```

Figura 1: Registro de la ruta home/{usuario} con su argumento de navegación

3. Navegar hacia HomeScreen al iniciar sesión

Cuando las credenciales ingresadas coinciden con alguna cuenta registrada, LoginScreen invoca onLoginExitoso(usuario). En MainActivity esa función navega a la ruta home pasando el usuario como argumento de la ruta:

```
composable("login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            navController.navigate("home/$usuario")
        },
        onIrARegistro = {
            navController.navigate("registro")
        }
    )
}
```

4. Crear el Composable HomeScreen

HomeScreen recibe el parámetro usuario: String y lo muestra en un Text con el mensaje "Bienvenido \$usuario":

```
Usage
@Composable
fun HomeScreen(usuario: String) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding( all = 24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Bienvenido $usuario",
            style = MaterialTheme.typography.headlineSmall
        )
    }
}
```

Figura 2: Función Composable HomeScreen(usuario)

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

5. Ejecutar y verificar

Al ejecutar el proyecto se muestra la pantalla de login. Al ingresar con el usuario admin y la contraseña 1234, la aplicación navega a la pantalla Home y muestra el mensaje de bienvenida con el nombre del usuario:

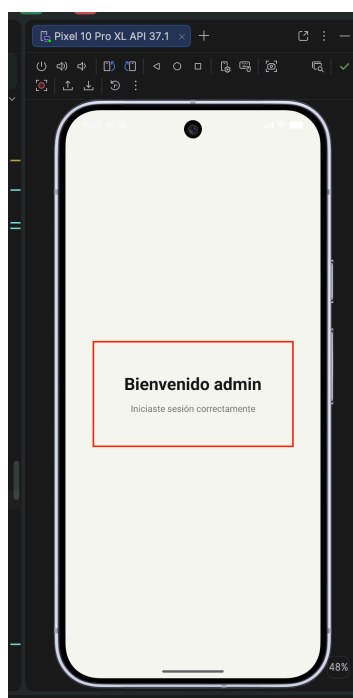


Figura 3: Pantalla de bienvenida con el nombre del usuario

Ejercicio 2: En RegistroScreen, al presionar “Aceptar”, guarda los datos ingresados (usuario y contraseña) en un archivo de texto plano cuentas.txt en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_APPEND), y regresa a LoginScreen mostrando un mensaje de confirmación.

En este ejercicio se persiste cada cuenta nueva en el archivo cuentas.txt del almacenamiento interno y se confirma la creación con el mensaje “Cuenta creada correctamente”.

1. Nuevos imports

Se agregan Context (para openFileOutput), Toast (mensaje de confirmación) y LocalContext (para obtener el contexto dentro del Composable):

```
import android.content.Context
import android.widget.Toast
import androidx.compose.ui.platform.LocalContext
```

2. Función guardarCuenta

La función abre cuentas.txt en modo MODE_APPEND y escribe una línea por cuenta con el formato usuario,password:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

1 Usage
fun guardarCuenta(context: Context, usuario: String, password: String) {
    val linea = "$usuario,$password\n"
    context.openFileOutput( name = "cuentas.txt", mode = Context.MODE_APPEND).use {
        archivo -> archivo.write( b = linea.toByteArray())
    }
}

```

Figura 4: Función guardarCuenta(context, usuario, password)

3. Cambios en RegistroScreen

Dentro del Composable se obtiene el contexto con `LocalContext.current` y el botón "Aceptar" ahora llama a `guardarCuenta(...)` antes de ejecutar `onRegistroExitoso()`:

```

val context = LocalContext.current

Button(
    onClick = {
        guardarCuenta(context, usuario, password)

        onRegistroExitoso()
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("Aceptar")
}

```

4. Mensaje de confirmación en MainActivity

`onRegistroExitoso` muestra un Toast con "Cuenta creada correctamente" antes de regresar a `LoginScreen` con `popBackStack()`:

```

onRegistroExitoso = {
    Toast.makeText(
        this,
        "Cuenta creada correctamente",
        Toast.LENGTH_SHORT
    ).show()

    navController.popBackStack()
},

```

5. Verificación

Al presionar "Aceptar" por cada cuenta, las líneas quedan en el archivo `cuentas.txt`:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

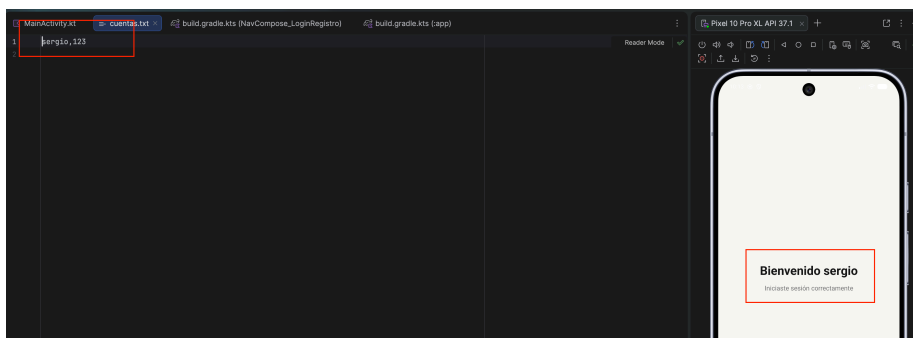


Figura 5: Archivo cuentas.txt con las cuentas guardadas

Ejercicio 3: En LoginScreen, al presionar “Ingresar”, busca una coincidencia en el archivo cuentas.txt (en lugar de la lista en memoria usada en el ejercicio resuelto). Si existe, navega a HomeScreen; si no, muestra el mensaje “Cuenta no encontrada”.

En este ejercicio la validación del ingreso ya no usa la lista en memoria cuentasRegistradas, sino que lee el archivo cuentas.txt para verificar si existe la cuenta usuario,password.

1. Función buscarCuenta

La función lee cuentas.txt con openFileInput y verifica si existe una línea que coincida exactamente con usuario,password. Si el archivo todavía no existe, devuelve false sin generar un error:

```
1 Usage
fun buscarCuenta(context: Context, usuario: String, password: String): Boolean {
    return try {
        val contenido = context.openFileInput(name = "cuentas.txt")
            .bufferedReader()
            .use { it.readText() }

        contenido.lineSequence().any { linea ->
            linea.trim() == "$usuario,$password"
        }
    } catch (e: Exception) {
        false
    }
}
```

Figura 6: Función buscarCuenta(context, usuario, password)

2. Cambios en LoginScreen

Dentro del Composable se obtiene el contexto con LocalContext.current:

```
val context = LocalContext.current
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

3. Botón “INGRESAR”

En lugar de buscar en cuentasRegistradas, el botón ahora llama a buscarCuenta(context, usuario, password). Si coincide, navega a home/\$usuario; si no, muestra “Cuenta no encontrada”:

```
val coincide = buscarCuenta(context, usuario, password)

if (coincide) {
    mensajeError = ""
    onLoginExitoso(usuario)
} else {
    mensajeError = "Cuenta no encontrada"
}
```

4. Verificación

Si la cuenta no existe en cuentas.txt, la pantalla de login muestra el mensaje de error:

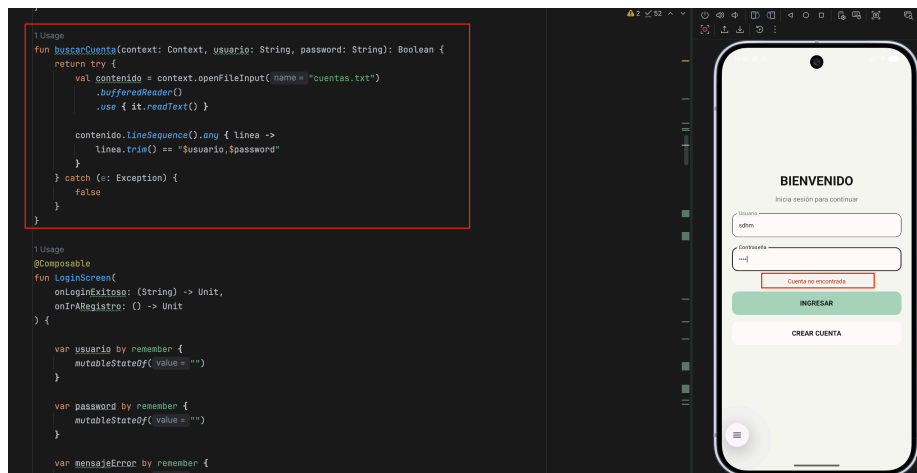


Figura 7: Mensaje “Cuenta no encontrada” al intentar ingresar con una cuenta inexistente

Ejercicio 4: (Reto opcional) Agrega validación de campos vacíos en ambos formularios, mostrando un mensaje de error si el usuario intenta enviar el formulario incompleto.

En este ejercicio se agrega validación de campos vacíos antes de guardar o buscar la cuenta: si falta algún dato, se muestra un mensaje y no se continúa con la operación.

1. Validación en RegistroScreen

Se agregó el estado mensajeError junto con su bloque de visualización (antes del botón “Aceptar”):

```
var mensajeError by remember {
    mutableStateOf("")
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

}

if (mensajeError.isNotEmpty()) {
    Text(
        text = mensajeError,
        color = MaterialTheme.colorScheme.error
    )

    Spacer(
        modifier = Modifier.height(8.dp)
    )
}

```

El botón “Aceptar” aplica la misma validación: solo guarda y navega atrás cuando ambos campos tienen contenido, y limpia mensajeError al registrar la cuenta correctamente:

```

Button(
    onClick = {
        when {
            usuario.isBlank() && password.isBlank() -> {
                mensajeError = "Completa todos los campos"
            }
            usuario.isBlank() -> {
                mensajeError = "El campo usuario es obligatorio"
            }
            password.isBlank() -> {
                mensajeError = "El campo contraseña es obligatorio"
            }
            else -> {
                val coincide = buscarCuenta(
                    context, usuario, password
                )
            }
        }
    }
)

```

Figura 8: Código de validación de campos vacíos

2. Verificación

Con la validación implementada, al dejar campos vacíos se muestra el mensaje correspondiente y no se realiza la búsqueda ni el guardado:

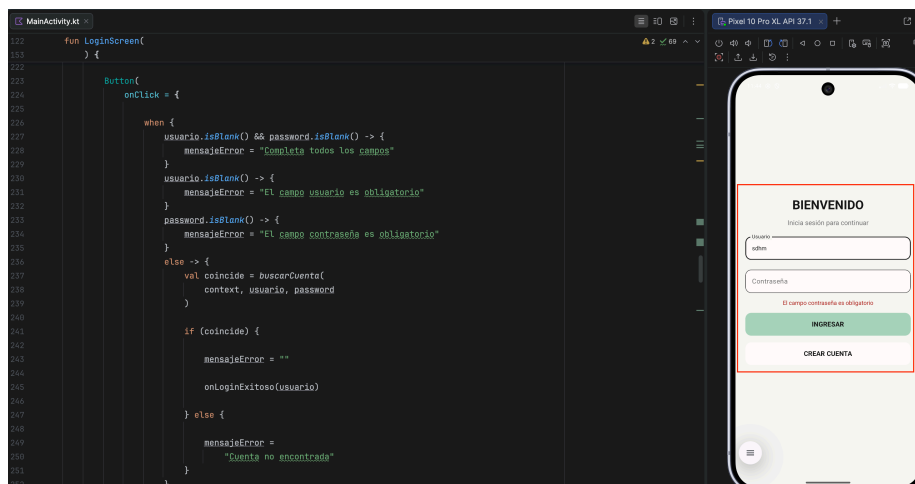


Figura 9: Mensaje de validación mostrado al enviar el formulario incompleto

CUESTIONARIO

CUESTIONARIO

Pregunta 1: ¿Qué ventajas tiene usar Navigation Compose (una sola Activity con varias pantallas) frente al enfoque tradicional de múltiples Activities comunicadas por Intent?

Las principales ventajas son:

- Transiciones más fluidas: al no destruir y recrear Activities, las animaciones entre pantallas son más rápidas y se pueden personalizar.
- Compartición de estado: el estado se comparte fácilmente entre pantallas ya que todas comparten el mismo scope de la Activity.
- Back stack más flexible: Navigation gestiona la pila de retroceso de forma más controlable.
- Patrón unificado: todo se desarrolla en Compose, sin alternar entre intents y callbacks.
- Mejor rendimiento: no hay sobrecarga de crear o destruir múltiples Activities.

Pregunta 2: Además de los argumentos de navegación, ¿qué otro mecanismo podrías usar para compartir datos entre pantallas en una app Compose (por ejemplo, un ViewModel compartido)? Explica brevemente cómo funcionaría.

Un ViewModel compartido es el mecanismo principal. Se crea un ViewModel único y se comparte entre pantallas a través del compositor usando un scope compartido como la Activity:

```
val viewModel: MiViewModel = viewModel()
```

Ventajas:

- Estado centralizado: toda la información compartida se encuentra en un solo lugar, evitando inconsistencias.
- Persistencia: el ViewModel sobrevive a cambios de configuración como rotaciones de pantalla.
- Desacoplamiento: las pantallas no necesitan conocerse entre sí; solo acceden al ViewModel.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

- Reactividad: al usar State o LiveData dentro del ViewModel, los cambios se reflejan automáticamente en todas las pantallas que observen esos estados.

CONCLUSIONES

CONCLUSIONES

Conclusión 1

Contenido de la conclusión 1...

REFERENCIAS

Bibliography